
ENGINEERING SPECIFICATION

Technical Architecture & Build Specification

A complete, implementation-ready elaboration of the Discipleship Pathway ecosystem — system architecture, relational schema, API surface, infrastructure topology, and security model.

Prepared for

Global Church Leadership & Engineering Teams

Companion to

Ecosystem Blueprint & PRD v1.0

0. Document Control & How to Read This

This document is the engineering elaboration of the *Nuru Place Discipleship Pathway Ecosystem Blueprint & PRD (v1.0)*. The PRD answers **what** and **why**; this specification answers **how**. Where the PRD left an interface implied (the cut-off schema, the giving ledger, the sync engine, the security posture), this document closes the gap with concrete, build-ready definitions.

0.1 Scope of this document

Five deliverables are treated end-to-end and are designed to be internally consistent — the schema in §2 is the contract the API in §3 exposes, deployed on the infrastructure in §4, and constrained by the security model in §5.

Deliverable	What it locks down	Section
System Architecture	Tier model, microservice boundaries, offline-first sync engine, the engagement-scoring (Ei) pipeline, gating logic, and the event flows behind the two user journeys.	§1
Database Schema	Complete 3NF PostgreSQL DDL across all domains, indexing & partitioning strategy, materialized views, functions, and migration plan.	§2
API Specification	REST conventions, the full endpoint catalog by domain, error taxonomy, idempotency, the offline batch-sync contract, and webhooks.	§3
Infrastructure Design	Region strategy for African last-mile latency, network topology, orchestration, data tier, CDN, CI/CD, observability, scaling, and DR.	§4
Security Model	Threat model, authN/authZ, encryption & key management, PCI posture, device/offline security, privacy & compliance (Kenya DPA), and incident response.	§5

0.2 Decisions made on top of the PRD

The PRD fixed the stack (React Native, Node/TypeScript microservices, 3NF PostgreSQL, Redis, SQLite, Cloudinary, Stripe, YouVersion, KingsChat). This document does not relitigate those. It does make the following **net-new engineering decisions**, flagged here for leadership visibility:

Decision	Rationale
Engagement score recomputed by a nightly batch job + incremental triggers , persisted to a snapshot table.	A 30-day sliding window over hundreds of thousands of members is too expensive to compute per request; snapshots make cohort sorting an indexed read.
Offline sync uses a per-record version + server-authoritative merge with an idempotent client mutation queue.	Volatile cellular networks (a core PRD constraint) require deterministic conflict resolution, not last-writer-wins guessing.
Money never touches our servers — Stripe Elements tokenization, PCI-DSS SAQ-A.	Keeps the platform out of cardholder-data scope entirely; only Stripe tokens and our own ledger are stored.
A single transactional outbox table drives all asynchronous side-effects (notifications, certificate rendering, score recompute triggers).	Guarantees exactly-once intent even if a worker crashes mid-process; avoids dual-write inconsistency between DB and queue.
Region anchored in af-south-1 (Cape Town) with edge caching, not a US/EU primary.	The membership is global but heavily African; primary write latency should not cross an ocean for the largest user segment.

0.3 Open assumptions & questions for sign-off

These are listed in full in Appendix C, but the load-bearing ones are: (a) expected concurrency is read-heavy with bursty write spikes around service times; (b) KingsChat SSO issues an OIDC-compatible token we can validate; (c) certificate "immutability" means tamper-evident (signed + verifiable), not blockchain-anchored. None of these block the build — they shape it.

Contents

1 System Architecture

1.1 Architectural principles · 1.2 Topology · 1.3 Client tier · 1.4 Edge & gateway
1.5 Service decomposition · 1.6 Data tier · 1.7 Offline-first sync engine
1.8 Engagement (Ei) pipeline · 1.9 Gating engine · 1.10 Key flows · 1.11 Service contracts

2 Database Schema

2.1 Principles · 2.2 Full DDL (all domains) · 2.3 Indexing · 2.4 Partitioning
2.5 Materialized views & functions · 2.6 Migrations & seeding

3 API Specification

3.1 Conventions · 3.2 Error catalog · 3.3 Endpoint catalog · 3.4 Examples
3.5 Webhooks · 3.6 Offline sync contract · 3.7 OpenAPI fragment

4 Infrastructure Design

4.1 Region strategy · 4.2 Network · 4.3 Orchestration · 4.4 Data tier · 4.5 Media/CDN
4.6 CI/CD · 4.7 Observability & SLOs · 4.8 Scaling · 4.9 DR · 4.10 Environments

5 Security Model

5.1 Posture · 5.2 Threat model · 5.3 AuthN · 5.4 AuthZ · 5.5 Data protection
5.6 Payments · 5.7 Device/offline · 5.8 API hardening · 5.9 Privacy & minors · 5.10 IR

A–C Appendices: capacity math · configuration · assumptions

1. System Architecture

The architecture is organised around one hard constraint from the PRD — **offline-first behaviour on volatile networks** — and one analytical promise — a **defensible, recomputable engagement score**. Everything else (microservice boundaries, the gateway, the data tier) is shaped to serve those two without sacrificing the high-transaction scalability target.

1.1 Architectural principles

Principle	Implication for the build
Offline is the default, not a fallback	The mobile client is a first-class system of record for in-flight work. The server is authoritative on conflict, but the client must function fully with zero connectivity and reconcile later (§1.7).
Server-authoritative truth	Gating decisions, exam pass/fail, score computation, and money are decided server-side. The client may <i>optimistically</i> reflect them but never originates them.
Stateless services, stateful data	Every microservice is horizontally scalable and holds no session in process; all state lives in PostgreSQL / Redis. This is what makes the high-concurrency target reachable.
Idempotency everywhere money or sync touches	Payment webhooks, certificate issuance, and the sync mutation queue all carry idempotency keys so retries on a flaky network cannot double-apply.
Read/write asymmetry	Reads vastly outnumber writes (dashboards, scripture, video). Reads are served from replicas, caches, and CDN; writes go to the primary and are kept small.
Separation of metrics from ministry	Per the PRD scope boundary, the platform automates measurement and delivery; it never models counselling or pastoral relationship as data beyond the multiplier → disciple tree.

1.2 High-level topology

Four tiers, with a clear cut between the public edge and the private service mesh. Nothing in the service or data tier is reachable from the public internet except through the gateway.

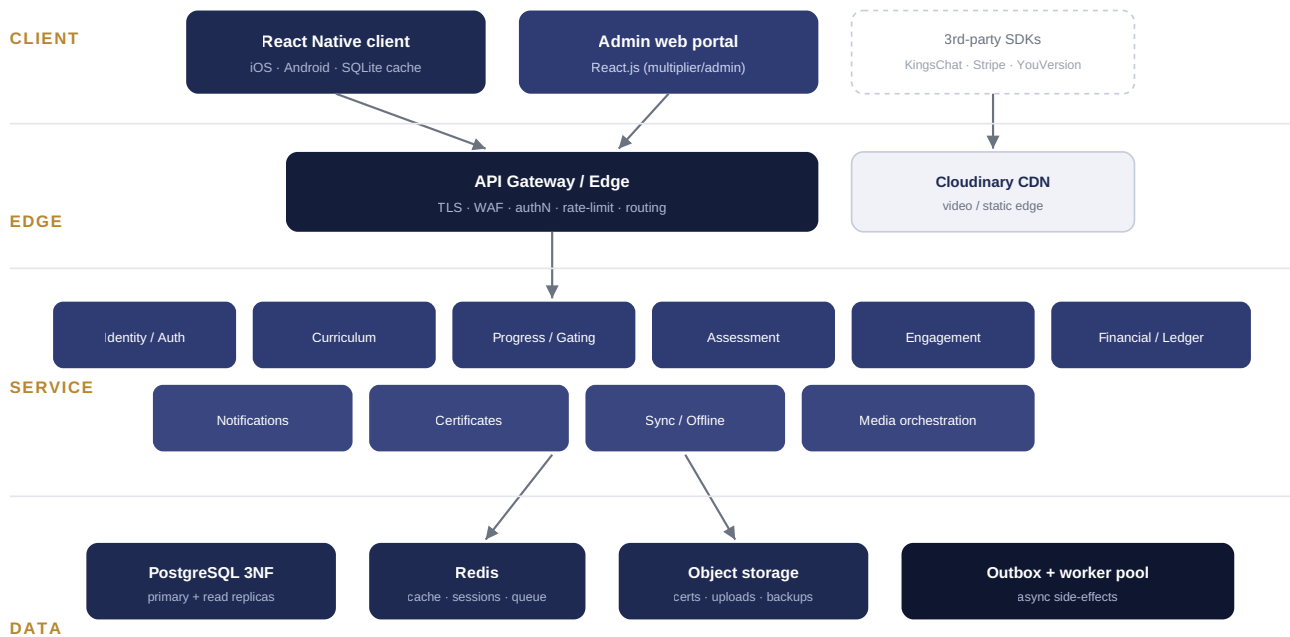


Figure 1 — Four-tier topology. Public traffic terminates at the gateway; the service mesh and data tier are private. Media bypasses services entirely via the CDN.

1.3 Client tier

Mobile client (React Native, TypeScript)

The mobile app is structured exactly as the PRD source tree implies: a Redux Toolkit store with an `offlineSlice` that queues mutations and a `userSlice` for session context, presentational components decoupled from network state, and an Axios client that injects the JWT. Three architectural details matter beyond that tree:

- **Local-first reads.** Curriculum content, the active module, quiz banks for unlocked modules, and recently-read scripture are cached in encrypted SQLite. The dashboard renders entirely from local state on launch, then reconciles in the background — the user never stares at a spinner because a tower dropped.
- **Mutation queue, not request mirror.** Offline actions (lesson completion, quiz submission, attendance scan, habit ticks) are written to a local `pending_mutations` table as intent records with a client-generated UUID and a monotonic sequence. They are replayed in order when connectivity returns (§1.7, §3.6).
- **Video isolation.** Per PRD §7.1, the `VideoPlayer` component is a decoupled state controller — it emits coarse playback events (`started` , `paused` , `completed_75pct`) rather than streaming continuous telemetry, which prevents re-render storms when the network flips between cell and Wi-Fi.

Admin web portal (React.js)

The portal is online-only (multipliers work from offices and on Wi-Fi) and read-optimised. Its defining screen is the cohort table sorted ascending by engagement score; that sort is a single indexed query against the snapshot table (§2.5), not a live computation. It also hosts the reflection-review queue, the relationship-tree editor, and the curriculum rich-text editors.

1.4 Edge & API gateway

A single ingress concentrates the cross-cutting concerns so individual services stay thin:

Concern	Handled at the edge
TLS termination	TLS 1.2+ only; HSTS; certificates from a managed CA, auto-rotated.
Authentication	JWT signature + expiry validated at the gateway; identity claims forwarded as signed internal headers so services never re-parse tokens.
Rate limiting	Token-bucket per IP and per user; stricter buckets on auth, payment, and sync endpoints (\$5.8).
WAF	OWASP core rule set; payload size caps; basic bot mitigation on auth routes.
Routing	Path-based routing to services; canary weighting for progressive rollout.
Observability	Injects a <code>X-Request-Id</code> correlation header propagated through the mesh and into logs/traces.

1.5 Service decomposition

Services are split by **bounded context and rate of change**, not by table. Curriculum changes weekly and is read-heavy; financial changes rarely but must be auditable; engagement is compute-heavy and batch-driven. Splitting them lets each scale and deploy on its own cadence.

Service	Owns	Key dependencies
Identity / Auth	Users, roles, sessions, refresh tokens, KingsChat/Google/Apple OAuth exchange, RBAC checks.	PostgreSQL (users, sessions), Redis (session lookup), KingsChat SSO.
Curriculum	Levels, modules, lesson content, question banks, pass marks, curriculum versioning.	PostgreSQL (read-mostly), Redis cache, Cloudinary (video URLs).
Progress / Gating	Enrollments, module progress, the sequential unlock engine, level transitions.	PostgreSQL, Assessment (pass signals), Certificates (graduation trigger).
Assessment	Randomised quiz assembly, automated scoring, attempt logs, reflection submission & review queue.	Curriculum (question bank), Progress (writes pass/fail).
Engagement	The E _i pipeline: window aggregation, snapshot table, cohort ranking reads.	PostgreSQL replica + materialized views; scheduled worker.
Financial / Ledger	Giving, tithes, offerings, media purchases, Stripe orchestration, double-entry ledger, idempotent webhooks.	Stripe, PostgreSQL (ledger + processed_webhooks).
Notifications	Push/email scheduling, the 12-nudge cadence, inactivity triggers, quiet-hours by local timezone.	Outbox, Redis (schedules), APNs/FCM, email provider.
Certificates	Tamper-evident PDF generation, verification-code issuance, public verify endpoint.	Object storage, Progress (graduation events), signing key (KMS).
Sync / Offline	Batch pull (deltas) and batch push (mutation replay) for mobile clients; conflict resolution.	All write-owning services via internal calls; PostgreSQL change log.
Media orchestration	Signed Cloudinary URLs, transcode requests, adaptive-bitrate manifest brokering.	Cloudinary API.

DEPLOYMENT NOTE

These are logical boundaries. They can ship as separate deployables from day one, or as a modular monolith with these module seams and be split later — the seams are what matter. The schema, API, and security models in this document hold either way.

1.6 Data tier

- **PostgreSQL (3NF)** is the single source of truth. One primary takes writes; read replicas serve dashboards, curriculum, and engagement reads. Connection pooling (PgBouncer) sits in front so thousands of stateless service instances do not exhaust backend connections.
- **Redis** serves three distinct roles kept on separate logical databases: low-latency session/identity lookups, a hot cache for curriculum and computed dashboard fragments, and a delayed-job queue for the notification cadence.
- **Object storage** holds rendered certificate PDFs, user-uploaded artefacts, and database backups — never application secrets.
- **Transactional outbox** (a PostgreSQL table drained by a worker pool) is how a write and its side-effects stay consistent: the business row and the outbox row commit in one transaction; the worker then publishes the side-effect at-least-once and the consumer dedupes by event id.

1.7 Offline-first sync engine

This is the most consequential subsystem in the platform and the one the PRD leaves most implicit. The design goal: a member can complete lessons, take quizzes, tick habits, and scan attendance for days with no connectivity, then have all of it reconcile deterministically — never silently lost, never double-applied, with the server winning any genuine conflict.

Sync model

Sync is split into two independent flows so a slow push never blocks a fast pull:

Flow	Direction	Mechanism
Pull (delta)	Server → client	Client sends its last-seen <code>cursor</code> per domain; server returns only rows changed since, plus tombstones for deletions, plus a new cursor.
Push (replay)	Client → server	Client replays its ordered <code>pending_mutations</code> queue. Each carries a client UUID idempotency key; the server applies, validates against authoritative rules, and returns a permutation result.

Conflict resolution rules

Last-writer-wins is forbidden for anything meaningful. Resolution is by record class:

Record class	Resolution policy
Append-only events (attendance scans, habit ticks, quiz attempts)	No conflict possible. Each is a new immutable row keyed by client UUID; replaying twice is a no-op via the idempotency key.
Progress state (module completion)	Monotonic merge. Completion only moves forward; a stale client claiming "incomplete" never overwrites a server "complete".
Profile fields (phone, cell group)	Server-authoritative with version check. Client sends the <code>row_version</code> it edited; mismatch returns <code>409 CONFLICT</code> with the current server value for the client to re-merge.
Gating / level transitions	Server-only. Clients cannot originate; they receive the result on pull. A client that thinks it unlocked Level 2 offline is corrected if the exam was not actually passed server-side.
Money	Never queued offline. Payments require connectivity; the client blocks the giving flow when offline rather than queuing financial intent.

Mutation lifecycle

```
// client: pending_mutations (local SQLite)
{ mutation_id: "cla9-...uuid", // idempotency key, client-generated
  seq: 482, // monotonic per-device order
  domain: "module_progress",
  op: "complete",
  payload: { module_id: "...", completed_at: "2026-06-04T19:11:00+03:00" },
  status: "pending" } // pending → acked → applied | rejected

// server: POST /v1/sync/push -> per-mutation result
{ results: [
  { mutation_id: "cla9-...", status: "applied", server_version: 7 },
  { mutation_id: "b3f2-...", status: "duplicate", note: "already processed" },
  { mutation_id: "9d77-...", status: "rejected", code: "GATE_LOCKED",
    detail: "module 4 not yet unlocked" }
] }
```

Rejected mutations are surfaced to the user with a human-readable reason and removed from the queue; duplicates are dropped silently. The queue is drained in `seq` order and never reordered, so causal dependencies (complete module 3 → then module 4) hold.

WHY A PER-RECORD VERSION, NOT A GLOBAL ONE

A single global version vector would force the entire client state to refuse a push if any one row was stale. Per-record versioning lets 99% of a member's offline work commit cleanly while only the genuinely conflicting profile edit bounces back for re-merge.

1.8 Engagement scoring (E_i) pipeline

The PRD defines the score but not how it is produced at scale. Computing a 30-day sliding window per member on every dashboard load would melt the database. The pipeline therefore **pre-computes and persists** scores into a snapshot table that the cohort view reads with a single indexed query.

$$E_i = w_h \cdot H_i + w_c \cdot C_i + w_a \cdot A_i \quad (w_h=0.40, \quad w_c=0.35, \quad w_a=0.25)$$

Component normalisation (all clamped to [0,1])

Term	Definition	Normalisation (build-ready)
H_i Habits	30-day interaction execution rate: lesson opens + scripture-read interactions.	<code>min(1, active_days_30d / 20)</code> — 20 active days in 30 is treated as a full habit score, so the metric saturates rather than rewarding obsessive logging.
C_i Curriculum	Fraction of the 45-module framework completed.	<code>modules_completed / 45</code> — cumulative, slow-moving; it anchors the score so a fully-progressed member is not flagged as stalling on a quiet week.
A_i Attendance	Verified QR check-ins in the trailing 30 days.	<code>min(1, checkins_30d / expected_30d)</code> where <code>expected_30d</code> derives from the member's cell cadence (default ≈ 8 : weekly cell + Sunday).

Computation strategy

- **Nightly batch (authoritative).** A scheduled worker runs per region during local low-traffic hours, recomputing E_i for every active enrollment from the replica and upserting into `engagement_scores`. This is the number the cohort table sorts on.
- **Incremental nudge (freshness).** When a high-signal event lands (a missed-session threshold crossed, a 14-day habit gap), the outbox triggers a single-member recompute so a stalling member surfaces before the next nightly run rather than after it.
- **Banding.** Each snapshot stores both the raw score and a band (`thriving` ≥ 0.75 , `steady` $0.55\text{--}0.74$, `watch` $0.40\text{--}0.54$, `at_risk` < 0.40) so the portal can colour-code and filter without re-deriving thresholds client-side.

The reference aggregation query and the snapshot table are specified in §2.5. The example in the PRD — a member dropping to $E_i = 0.45$ after a 14-day gap — falls in the *watch* band and is exactly the case the incremental trigger is designed to surface early.

1.9 Gating engine

The gating engine is a pure server-side function over progress state. It never trusts the client. Three rules compose:

1. **Sequential modules.** Module `n` within a level unlocks only when module `n-1` is marked complete *and* its quiz passed at or above `quiz_pass_mark`.
2. **Level exam.** The final module of a level unlocks the level exam; passing at or above `required_exam_pass_mark` (default 80%) is necessary but not sufficient to advance.
3. **Reflection gate.** Level advancement additionally requires an approved reflection — a human pastor clicks "Approve" in the review queue. Only then does the engine flip `enrollments.current_level` and the Certificates service issue the credential.

HARD LOCK INVARIANT

Higher levels are *hard-locked*: there is no API path that returns Level 2 module content for a member whose `current_level` is 1, regardless of client claims. The gating check runs in the Progress service on every content fetch, not just on transition.

1.10 Key flows

Flow A — Module completion → quiz → unlock (offline-tolerant)

```
Client      Sync      Progress      Assessment      Curriculum
| complete mod ----> queue (offline ok)
| submit quiz ----> queue
|
|              | (connectivity returns) push replay -->|
|              |----- apply completion ----->|
|              |----- score attempt ----->|
|              | <-- pass 84% --
|              |<----- unlock module n+1 -----|
|<-- pull delta: module n+1 now unlocked, content keys -----|
```

Flow B — Level graduation → reflection → certificate

```
Member passes Level exam (≥80%) -> submits reflection text
    -> reflection enqueued to reflection_reviews
Pastor opens portal review queue -> clicks Approve
    -> Progress flips current_level 1->2 (single tx + outbox row)
    -> Outbox worker calls Certificates service
    -> Cert service renders signed PDF + verification code
    -> stored in object storage; member notified; Level 2 unlocked
```

Flow C — Giving payment (idempotent, money never offline)

```
Client (online only) -> Stripe Elements tokenizes card (card data never hits our server)
    -> POST /v1/giving/intents { amount, fund, idempotency_key }
    -> Financial service creates PaymentIntent via Stripe
    -> client confirms with Stripe -> Stripe fires webhook
    -> Gateway -> Financial: verifyStripeWebhookIdempotent (HMAC + row-lock dedupe)
    -> ledger double-entry posted in one tx + outbox receipt
    -> member sees confirmed giving record on next read
```

1.11 Inter-service communication & contracts

- **Synchronous (request/response):** internal REST/gRPC over the private mesh for read-time needs — e.g. Progress asking Assessment for a pass signal. Calls are timeout-bounded with circuit breakers so one slow service cannot cascade.
- **Asynchronous (events):** anything that can be eventually consistent goes through the outbox — certificate issuance, notification scheduling, engagement recompute. Consumers are idempotent and dedupe on `event_id`.
- **Contracts:** every internal and external interface is described in a shared OpenAPI/Protobuf registry, versioned, and validated in CI so a breaking change fails the build, not production.

2. Database Schema

This is the full 3NF PostgreSQL contract. It completes the eleven tables the PRD began (including the truncated `processed_webhooks`) and adds the domains the PRD specified in prose but never modelled: giving & the double-entry ledger, certificates, notifications & device tokens, offline sync bookkeeping, RBAC scoping, and audit/outbox. Every API resource in §3 maps to tables defined here.

2.1 Design principles

Principle	Applied as
Third normal form	No transitive dependencies; <code>home_cell_group</code> and funds are normalised into their own tables (the PRD's denormalised varchar is replaced by a FK).
UUID primary keys	<code>gen_random_uuid()</code> everywhere except the small fixed <code>levels</code> dimension, which keeps integer structural IDs 1–5.
Surrogate idempotency keys	Client-supplied UUIDs on every offline-originated insert, enforced <code>UNIQUE</code> , so replays are safe.
Soft delete + audit	<code>deleted_at</code> on user-facing entities; an append-only <code>audit_log</code> records who changed what.
Money as integer minor units	All amounts are <code>BIGINT</code> in the currency's minor unit (cents) — never floats — with an explicit ISO currency code.
Timestamps in UTC	<code>TIMESTAMPTZ</code> stored UTC; the client renders in the member's local timezone (also persisted for the nudge cadence).

2.2 Full DDL

Extensions, enums & organisational structure

```
CREATE EXTENSION IF NOT EXISTS pgcrypto;      -- gen_random_uuid()
CREATE EXTENSION IF NOT EXISTS pg_trgm;       -- fuzzy search on names/curriculum

CREATE TYPE user_role      AS ENUM ('Student','Instructor','Admin','SuperAdmin');
CREATE TYPE question_type AS ENUM ('MultipleChoice','TrueFalse','FillInTheBlank');
CREATE TYPE enrollment_state AS ENUM ('active','paused','completed','withdrawn');
CREATE TYPE review_state  AS ENUM ('pending','approved','rejected');
CREATE TYPE txn_status    AS ENUM ('requires_action','processing','succeeded','failed','refunded');
CREATE TYPE ledger_side   AS ENUM ('debit','credit');
CREATE TYPE notif_channel AS ENUM ('push','email');
CREATE TYPE notif_status  AS ENUM ('scheduled','sent','failed','suppressed');
CREATE TYPE engagement_band AS ENUM ('thriving','steady','watch','at_risk');
CREATE TYPE outbox_status AS ENUM ('pending','processing','done','dead');

-- Congregations / branches (multi-tenant root for a global church)
CREATE TABLE congregations (
  congregation_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  name            VARCHAR(255) NOT NULL,
  country        CHAR(2)      NOT NULL,          -- ISO 3166-1 alpha-2
  timezone       VARCHAR(64)  NOT NULL DEFAULT 'Africa/Nairobi',
  created_at     TIMESTAMPTZ  NOT NULL DEFAULT now()
);

-- Cell groups (normalises the PRD's home_cell_group varchar)
CREATE TABLE cell_groups (
  cell_group_id   UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  congregation_id UUID NOT NULL REFERENCES congregations(congregation_id),
  name           VARCHAR(150) NOT NULL,
  leader_user_id  UUID,          -- FK added after users exists
  meeting_cadence INT NOT NULL DEFAULT 8, -- expected check-ins / 30d (A1 baseline)
  created_at     TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (congregation_id, name)
);
```

1-2 · Identity, auth & RBAC scoping

```
CREATE TABLE users (
  user_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  email            CITEXT UNIQUE,          -- nullable: SSO-only users may lack email
  password_hash    VARCHAR(255),          -- nullable for pure-SSO accounts (argon2id)
  full_name        VARCHAR(255) NOT NULL,
  phone_number     VARCHAR(32)  NOT NULL,
  date_of_birth    DATE          NOT NULL,
  year_of_salvation INT,
  is_baptized      BOOLEAN       NOT NULL DEFAULT FALSE,
  cell_group_id    UUID          REFERENCES cell_groups(cell_group_id),
  congregation_id  UUID          NOT NULL REFERENCES congregations(congregation_id),
  role             user_role     NOT NULL DEFAULT 'Student',
  timezone         VARCHAR(64)   NOT NULL DEFAULT 'Africa/Nairobi',
  locale           VARCHAR(12)   NOT NULL DEFAULT 'en',
  is_minor         BOOLEAN GENERATED ALWAYS AS (date_of_birth > (CURRENT_DATE - INTERVAL '18 years'))
STORED,
  created_at       TIMESTAMPTZ   NOT NULL DEFAULT now(),
  updated_at       TIMESTAMPTZ   NOT NULL DEFAULT now(),
  deleted_at       TIMESTAMPTZ
);
ALTER TABLE cell_groups
  ADD CONSTRAINT fk_cell_leader FOREIGN KEY (leader_user_id) REFERENCES users(user_id);

-- Federated identities (KingsChat / Google / Apple). One user, many providers.
CREATE TABLE oauth_identities (
  identity_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id          UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  provider         VARCHAR(40) NOT NULL,          -- 'kingschat' | 'google' | 'apple'
  provider_sub     VARCHAR(255) NOT NULL,          -- subject id from the IdP
  linked_at       TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (provider, provider_sub)
);

-- Refresh-token family for rotation + reuse detection (see §5.3)
CREATE TABLE refresh_tokens (
  token_id        UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id         UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  family_id       UUID NOT NULL,                  -- rotating chain
  token_hash      VARCHAR(255) NOT NULL,          -- sha-256 of the token, never the token
  device_id       UUID,
  expires_at      TIMESTAMPTZ NOT NULL,
  revoked_at      TIMESTAMPTZ,
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Granular admin scoping: which multiplier oversees which cohort/cell.
CREATE TABLE leader_assignments (
  assignment_id    UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  leader_user_id   UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  cell_group_id    UUID NOT NULL REFERENCES cell_groups(cell_group_id) ON DELETE CASCADE,
  assigned_at      TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (leader_user_id, cell_group_id)
);
```

3-4 · Curriculum (with versioning)

```
CREATE TABLE levels (  
  level_number INT PRIMARY KEY,           -- 1..5, fixed dimension  
  title        VARCHAR(255) NOT NULL,  
  theme        TEXT,  
  required_exam_pass_mark NUMERIC(5,2) NOT NULL DEFAULT 80.00  
);  
  
CREATE TABLE modules (  
  module_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  level_number   INT NOT NULL REFERENCES levels(level_number),  
  module_sequence_number INT NOT NULL,  
  title          VARCHAR(255) NOT NULL,  
  lesson_content TEXT NOT NULL,  
  video_url      VARCHAR(512),  
  evaluation_kind VARCHAR(60),           -- 'quiz' | 'attendance' | 'exit_exam' ...  
  estimated_minutes INT,  
  quiz_pass_mark NUMERIC(5,2) NOT NULL DEFAULT 70.00,  
  is_published   BOOLEAN NOT NULL DEFAULT FALSE,  
  current_version INT NOT NULL DEFAULT 1,  
  UNIQUE (level_number, module_sequence_number)  
);  
  
-- Immutable content history so editing curriculum never breaks past attempts.  
CREATE TABLE module_versions (  
  version_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  module_id       UUID NOT NULL REFERENCES modules(module_id) ON DELETE CASCADE,  
  version_number  INT NOT NULL,  
  lesson_content  TEXT NOT NULL,  
  edited_by       UUID REFERENCES users(user_id),  
  created_at      TIMESTAMPTZ NOT NULL DEFAULT now(),  
  UNIQUE (module_id, version_number)  
);  
  
CREATE TABLE question_bank (  
  question_id     UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  module_id       UUID NOT NULL REFERENCES modules(module_id) ON DELETE CASCADE,  
  q_type          question_type NOT NULL,  
  question_text   TEXT NOT NULL,  
  answer_options  JSONB,                -- localized choice arrays  
  correct_answer  TEXT NOT NULL,  
  difficulty_rating INT NOT NULL DEFAULT 1,  
  is_active       BOOLEAN NOT NULL DEFAULT TRUE  
);
```

5-7 · Enrollment, progress & the habit signal

```
CREATE TABLE enrollments (  
  enrollment_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id       UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,  
  current_level INT NOT NULL REFERENCES levels(level_number) DEFAULT 1,  
  state         enrollment_state NOT NULL DEFAULT 'active',  
  started_at    TIMESTAMPTZ NOT NULL DEFAULT now(),  
  completed_at  TIMESTAMPTZ,  
  UNIQUE (user_id)  
);  
  
CREATE TABLE module_progress (  
  progress_id   UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  enrollment_id UUID NOT NULL REFERENCES enrollments(enrollment_id) ON DELETE CASCADE,  
  module_id     UUID NOT NULL REFERENCES modules(module_id),  
  is_completed  BOOLEAN NOT NULL DEFAULT FALSE,  
  completed_at  TIMESTAMPTZ,  
  client_mutation_id UUID UNIQUE,           -- idempotent offline completion  
  row_version   INT NOT NULL DEFAULT 1,  
  UNIQUE (enrollment_id, module_id)  
);  
  
-- The Hi signal source: every lesson open / scripture read (append-only).  
CREATE TABLE interaction_events (  
  event_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id       UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,  
  kind          VARCHAR(40) NOT NULL,        -- 'lesson_open' | 'scripture_read' | 'video_75pct'  
  module_id     UUID REFERENCES modules(module_id),  
  occurred_at   TIMESTAMPTZ NOT NULL,        -- client local event time (UTC stored)  
  client_event_id UUID UNIQUE,              -- idempotency for offline replay  
  ingested_at   TIMESTAMPTZ NOT NULL DEFAULT now()  
) PARTITION BY RANGE (occurred_at);
```


7-8 · Assessment & reflection review

```
CREATE TABLE quiz_attempts (  
  attempt_id      UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  progress_id     UUID NOT NULL REFERENCES module_progress(progress_id) ON DELETE CASCADE,  
  score_achieved  NUMERIC(5,2) NOT NULL,  
  is_passed       BOOLEAN NOT NULL,  
  question_set    JSONB NOT NULL,                -- the randomized question_ids served  
  client_mutation_id UUID UNIQUE,  
  attempted_at    TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
  
CREATE TABLE quiz_attempt_answers (  
  answer_id       UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  attempt_id      UUID NOT NULL REFERENCES quiz_attempts(attempt_id) ON DELETE CASCADE,  
  question_id     UUID NOT NULL REFERENCES question_bank(question_id),  
  given_answer    TEXT NOT NULL,  
  is_correct      BOOLEAN NOT NULL  
);  
  
CREATE TABLE reflection_reviews (  
  review_id       UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id         UUID NOT NULL REFERENCES users(user_id),  
  level_number    INT  NOT NULL REFERENCES levels(level_number),  
  reflection_text  TEXT NOT NULL,  
  state           review_state NOT NULL DEFAULT 'pending',  
  reviewed_by     UUID REFERENCES users(user_id),  
  feedback_notes  TEXT,  
  submitted_at    TIMESTAMPTZ NOT NULL DEFAULT now(),  
  reviewed_at     TIMESTAMPTZ,  
  UNIQUE (user_id, level_number)  
);
```

9-10 · Relationship tree, events & attendance

```
CREATE TABLE relationship_tree (  
  tree_id          UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  multiplier_id    UUID NOT NULL REFERENCES users(user_id),  
  disciple_id      UUID NOT NULL UNIQUE REFERENCES users(user_id),  
  established_at   TIMESTAMPTZ NOT NULL DEFAULT now(),  
  CHECK (multiplier_id <> disciple_id)  
);  
  
CREATE TABLE events (  
  event_id         VARCHAR(100) PRIMARY KEY,          -- human/QR-friendly id  
  congregation_id  UUID NOT NULL REFERENCES congregations(congregation_id),  
  cell_group_id    UUID REFERENCES cell_groups(cell_group_id),  
  title            VARCHAR(255) NOT NULL,  
  occurs_at       TIMESTAMPTZ NOT NULL,  
  qr_secret        VARCHAR(255) NOT NULL              -- HMAC seed for scan tokens ($5)  
);  
  
CREATE TABLE attendance_logs (  
  attendance_id    UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id          UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,  
  event_id         VARCHAR(100) NOT NULL REFERENCES events(event_id),  
  client_scan_id   UUID UNIQUE,                        -- idempotent offline scan  
  checked_in_at    TIMESTAMPTZ NOT NULL DEFAULT now(),  
  UNIQUE (user_id, event_id)  
);
```

Engagement snapshot (the E_i output table)

```
CREATE TABLE engagement_scores (  
  user_id          UUID PRIMARY KEY REFERENCES users(user_id) ON DELETE CASCADE,  
  cell_group_id    UUID REFERENCES cell_groups(cell_group_id),  
  h_score          NUMERIC(4,3) NOT NULL,             --  $H_i \in [0,1]$   
  c_score          NUMERIC(4,3) NOT NULL,             --  $C_i \in [0,1]$   
  a_score          NUMERIC(4,3) NOT NULL,             --  $A_i \in [0,1]$   
  e_score          NUMERIC(4,3) NOT NULL,             --  $E_i$  composite  
  band            engagement_band NOT NULL,  
  window_end       DATE NOT NULL,  
  computed_at      TIMESTAMPTZ NOT NULL DEFAULT now()  
);  
-- Cohort table sorts on this; index makes "lowest score in my cells" instant.  
CREATE INDEX idx_engagement_cell_score ON engagement_scores (cell_group_id, e_score);
```

Financial — funds, transactions, double-entry ledger, products

```
CREATE TABLE funds (
  fund_id  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  code     VARCHAR(40) UNIQUE NOT NULL,          -- 'tithe' | 'offering' | 'general' | 'media'
  name     VARCHAR(150) NOT NULL,
  is_active BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE transactions (
  transaction_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id       UUID NOT NULL REFERENCES users(user_id),
  fund_id       UUID REFERENCES funds(fund_id),
  amount_minor  BIGINT NOT NULL CHECK (amount_minor > 0), -- cents
  currency      CHAR(3) NOT NULL,                    -- ISO 4217
  status        txn_status NOT NULL DEFAULT 'processing',
  stripe_payment_intent VARCHAR(255) UNIQUE,
  idempotency_key VARCHAR(255) UNIQUE NOT NULL,      -- client-supplied
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now(),
  settled_at    TIMESTAMPTZ
);

-- Double-entry: every transaction yields balanced debit/credit rows.
CREATE TABLE ledger_entries (
  entry_id       UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  transaction_id UUID NOT NULL REFERENCES transactions(transaction_id),
  account        VARCHAR(60) NOT NULL,              -- 'cash:stripe' | 'fund:tithe' ...
  side           ledger_side NOT NULL,
  amount_minor  BIGINT NOT NULL CHECK (amount_minor > 0),
  currency      CHAR(3) NOT NULL,
  created_at    TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE products (
  product_id  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  title       VARCHAR(255) NOT NULL,
  price_minor BIGINT NOT NULL,
  currency    CHAR(3) NOT NULL,
  media_asset_id UUID,                -- FK added below
  is_active   BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE purchases (
  purchase_id  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id      UUID NOT NULL REFERENCES users(user_id),
  product_id   UUID NOT NULL REFERENCES products(product_id),
  transaction_id UUID NOT NULL REFERENCES transactions(transaction_id),
  granted_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (user_id, product_id)
);

-- 11 · Webhook idempotency ledger (the PRD's truncated table, completed)
CREATE TABLE processed_webhooks (
  event_id     VARCHAR(255) PRIMARY KEY,          -- provider event id
  provider     VARCHAR(50) NOT NULL,              -- 'Stripe'
  payload_hash VARCHAR(64) NOT NULL,              -- sha-256 of body, audit
  processed_at TIMESTAMPTZ NOT NULL DEFAULT now()
);
```

Certificates, notifications, media, devices/sync, audit & outbox

```

CREATE TABLE certificates (
  certificate_id  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id        UUID NOT NULL REFERENCES users(user_id),
  level_number   INT  REFERENCES levels(level_number),    -- null = full-program
  verification_code VARCHAR(24) UNIQUE NOT NULL,         -- printed on the PDF
  pdf_object_key VARCHAR(512) NOT NULL,                  -- object storage path
  content_hash   VARCHAR(64) NOT NULL,                   -- tamper-evidence
  signature      TEXT      NOT NULL,                     -- detached sig (KMS key)
  issued_at      TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE push_tokens (
  token_id  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id   UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  platform  VARCHAR(12) NOT NULL,                -- 'ios' | 'android'
  token     VARCHAR(512) NOT NULL,
  is_active BOOLEAN NOT NULL DEFAULT TRUE,
  updated_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  UNIQUE (token)
);

CREATE TABLE notification_preferences (
  user_id        UUID PRIMARY KEY REFERENCES users(user_id) ON DELETE CASCADE,
  push_enabled   BOOLEAN NOT NULL DEFAULT TRUE,
  email_enabled  BOOLEAN NOT NULL DEFAULT TRUE,
  quiet_from     TIME NOT NULL DEFAULT '21:00', -- local time, no nudges after
  quiet_to       TIME NOT NULL DEFAULT '07:00',
  max_daily      INT  NOT NULL DEFAULT 3        -- caps the 12-nudge cadence
);

CREATE TABLE notifications (
  notification_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id        UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  channel        notif_channel NOT NULL,
  template       VARCHAR(80) NOT NULL,
  payload        JSONB NOT NULL,
  status         notif_status NOT NULL DEFAULT 'scheduled',
  scheduled_for  TIMESTAMPTZ NOT NULL,
  sent_at       TIMESTAMPTZ
);

CREATE TABLE media_assets (
  media_asset_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  cloudinary_id  VARCHAR(255) NOT NULL,
  kind           VARCHAR(40) NOT NULL,        -- 'lesson_video' | 'vod' | 'product'
  duration_sec   INT,
  created_at     TIMESTAMPTZ NOT NULL DEFAULT now()
);

ALTER TABLE products ADD CONSTRAINT fk_product_media
  FOREIGN KEY (media_asset_id) REFERENCES media_assets(media_asset_id);

-- Registered client devices + per-device sync cursors
CREATE TABLE client_devices (
  device_id  UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  user_id    UUID NOT NULL REFERENCES users(user_id) ON DELETE CASCADE,
  platform   VARCHAR(12) NOT NULL,
  app_version VARCHAR(20),
  last_seen_at TIMESTAMPTZ,
  sync_cursors JSONB NOT NULL DEFAULT '{}', -- { domain: last_seen_change_id }
);

-- Change log feeding delta-pull (one row per server-side mutation)

```

```

CREATE TABLE change_log (
  change_id BIGSERIAL PRIMARY KEY,
  domain    VARCHAR(40) NOT NULL,
  row_id    UUID,
  user_id   UUID,
  op        VARCHAR(10) NOT NULL,      -- scope: whose deltas this belongs to
  changed_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

CREATE TABLE audit_log (
  audit_id BIGSERIAL PRIMARY KEY,
  actor_id UUID,
  action    VARCHAR(80) NOT NULL,
  entity    VARCHAR(60) NOT NULL,
  entity_id VARCHAR(64),
  metadata  JSONB,
  occurred_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

-- Transactional outbox: business write + this row commit together
CREATE TABLE outbox (
  outbox_id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
  topic     VARCHAR(80) NOT NULL,      -- 'certificate.issue' | 'engagement.recompute' ...
  payload    JSONB NOT NULL,
  status     outbox_status NOT NULL DEFAULT 'pending',
  attempts   INT NOT NULL DEFAULT 0,
  available_at TIMESTAMPTZ NOT NULL DEFAULT now(),
  created_at TIMESTAMPTZ NOT NULL DEFAULT now()
);

```

2.3 Indexing strategy

Indexes are chosen from the actual query shapes in §1.10 and §3, not speculatively.

Index	Serves
<code>users(congregation_id, role)</code>	Admin listing members by branch and role.
<code>users USING gin(full_name gin_trgm_ops)</code>	Fuzzy member search in the portal.
<code>engagement_scores(cell_group_id, e_score)</code>	The cohort table — lowest score first — as one index range scan.
<code>module_progress(enrollment_id, is_completed)</code>	Gating checks and C _i counting.
<code>interaction_events(user_id, occurred_at DESC)</code> (per partition)	30-day H _i window aggregation.
<code>attendance_logs(user_id, checked_in_at DESC)</code>	30-day A _i window aggregation.
<code>reflection_reviews(state, submitted_at) WHERE state='pending'</code>	The pastor review queue (partial index).
<code>change_log(user_id, domain, change_id)</code>	Delta-pull "everything for this user since cursor".
<code>transactions(idempotency_key), processed_webhooks(event_id)</code>	Payment dedupe / replay protection.
<code>outbox(status, available_at) WHERE status='pending'</code>	Worker poll for due side-effects (partial index).

2.4 Partitioning for high-transaction tables

Two tables grow without bound and dominate write volume: `interaction_events` and (over time) `attendance_logs` and `change_log`. They are range-partitioned by month so that the 30-day window queries touch at most two partitions, and old partitions can be detached/archived cheaply.

```
-- Monthly partitions, created ahead of time by a maintenance job
CREATE TABLE interaction_events_2026_06 PARTITION OF interaction_events
  FOR VALUES FROM ('2026-06-01') TO ('2026-07-01');
CREATE TABLE interaction_events_2026_07 PARTITION OF interaction_events
  FOR VALUES FROM ('2026-07-01') TO ('2026-08-01');
-- pg_partman (or a cron function) keeps N+2 months provisioned and prunes >13 months.
```

2.5 Materialized views & functions

Engagement aggregation (the authoritative nightly recompute)

This is the query the batch worker runs per cohort and upserts into `engagement_scores`. It expresses E_i exactly as §1.8 normalises it.

```

WITH win AS (SELECT (CURRENT_DATE - INTERVAL '30 days') AS lo, CURRENT_DATE AS hi),
hab AS ( -- Hi: active days / 20, capped
  SELECT user_id,
    LEAST(1.0, COUNT(DISTINCT date_trunc('day', occurred_at)) / 20.0) AS h
  FROM interaction_events, win
  WHERE occurred_at >= win.lo GROUP BY user_id),
cur AS ( -- Ci: completed modules / 45
  SELECT e.user_id, LEAST(1.0, COUNT(*) FILTER (WHERE mp.is_completed) / 45.0) AS c
  FROM enrollments e JOIN module_progress mp USING (enrollment_id) GROUP BY e.user_id),
att AS ( -- Ai: checkins / cell cadence, capped
  SELECT u.user_id,
    LEAST(1.0, COUNT(al.*)::numeric / GREATEST(cg.meeting_cadence,1)) AS a
  FROM users u
  LEFT JOIN cell_groups cg ON cg.cell_group_id = u.cell_group_id
  LEFT JOIN attendance_logs al ON al.user_id = u.user_id
  AND al.checked_in_at >= (CURRENT_DATE - INTERVAL '30 days')
  GROUP BY u.user_id, cg.meeting_cadence)
SELECT u.user_id, u.cell_group_id,
  COALESCE(hab.h,0) h_score, COALESCE(cur.c,0) c_score, COALESCE(att.a,0) a_score,
  ROUND(0.40*COALESCE(hab.h,0) + 0.35*COALESCE(cur.c,0) + 0.25*COALESCE(att.a,0), 3) AS e_score
FROM users u
LEFT JOIN hab ON hab.user_id=u.user_id
LEFT JOIN cur ON cur.user_id=u.user_id
LEFT JOIN att ON att.user_id=u.user_id
WHERE u.deleted_at IS NULL AND u.role='Student';
-- result rows are upserted into engagement_scores with band derived by CASE on e_score.

```

Gating check (server-side, callable from Progress service)

```

CREATE OR REPLACE FUNCTION fn_module_unlocked(p_enrollment UUID, p_module UUID)
RETURNS BOOLEAN LANGUAGE sql STABLE AS $$
  WITH m AS (SELECT level_number, module_sequence_number FROM modules WHERE module_id=p_module),
  prev AS (SELECT module_id FROM modules, m
    WHERE modules.level_number=m.level_number
    AND modules.module_sequence_number=m.module_sequence_number-1)
  SELECT CASE
    WHEN (SELECT module_sequence_number FROM m)=1 THEN TRUE -- first module of a level
    ELSE EXISTS (SELECT 1 FROM module_progress mp JOIN prev USING(module_id)
      WHERE mp.enrollment_id=p_enrollment AND mp.is_completed)
  END;
$$;

```

2.6 Migrations & seeding

- **Forward-only, versioned migrations** (one timestamped SQL file per change) run in CI against an ephemeral database, then in staging, then production behind a maintenance gate. No destructive change ships without a paired, tested rollback note.
- **Seed data** loads the five `levels` and all 45 `modules` from the PRD curriculum appendix, the four core `funds` (tithe, offering, general, media), and a SuperAdmin bootstrap account created out-of-band (never committed).
- **Reference integrity check** in CI asserts the seed produces exactly 5 levels and 45 modules with contiguous sequence numbers — the gating engine depends on that contiguity.

3. API Specification

A versioned REST surface over the §2 schema. Every endpoint is stateless, authenticated at the gateway, and idempotent wherever it creates state from a flaky client. The catalog below is complete for v1; representative request/response bodies and the offline batch-sync contract follow.

3.1 Conventions

Concern	Convention
Base & versioning	<code>https://api.nuruplace.org/v1/...</code> — major version in the path; additive changes only within a major.
Auth	<code>Authorization: Bearer <access_jwt></code> . Short-lived access token (15 min) + rotating refresh token (§5.3).
Content	<code>application/json; charset=utf-8</code> . Times are ISO-8601 with offset. Money is <code>{ amount_minor, currency }</code> .
Idempotency	Unsafe creates accept <code>Idempotency-Key</code> header (or body <code>client_mutation_id</code>); replays return the original result, not a duplicate.
Pagination	Cursor-based: <code>?limit=50&cursor=...</code> ; response carries <code>{ data, next_cursor }</code> . No offset pagination on large tables.
Filtering / sort	Whitelisted query params only (e.g. <code>?sort=e_score&order=asc&band=at_risk</code>); arbitrary SQL-ish filters are rejected.
Correlation	Every response echoes <code>X-Request-Id</code> for support and tracing.
Rate limits	Returned as <code>X-RateLimit-Remaining / -Reset ; 429</code> on exhaustion (§5.8).

3.2 Standard error envelope & catalog

```
{ "error": {  
  "code": "GATE_LOCKED",  
  "message": "Module 4 is not yet unlocked.",  
  "request_id": "req_8fa3...",  
  "details": { "module_sequence_number": 4 } } }
```

HTTP	code	Meaning
400	VALIDATION_FAILED	Body or params failed schema validation; <code>details</code> lists fields.
401	AUTH_REQUIRED / TOKEN_EXPIRED	Missing/expired access token; client should refresh.
403	FORBIDDEN_SCOPE	Authenticated but not authorised for this resource (e.g. multiplier querying a cell outside their assignments).
404	NOT_FOUND	Resource absent or soft-deleted.
409	CONFLICT / VERSION_STALE	Optimistic-concurrency clash on a profile edit; current value returned.
409	GATE_LOCKED	Content or action blocked by the gating engine.
422	UNPROCESSABLE	Well-formed but semantically invalid (e.g. quiz submitted for an unowned attempt).
429	RATE_LIMITED	Bucket exhausted; <code>Retry-After</code> set.
500/503	INTERNAL / UPSTREAM_UNAVAILABLE	Server fault or a dependency (Stripe/KingsChat) down; client retries with backoff.

3.3 Endpoint catalog

Authentication & identity

Method	Path	Purpose
POST	/v1/auth/oauth/{provider}	Exchange a KingsChat/Google/Apple authorization code for our tokens; provisions the user on first login.
POST	/v1/auth/token/refresh	Rotate refresh token → new access+refresh pair; reuse detection revokes the family.
POST	/v1/auth/logout	Revoke the current refresh-token family.
GET	/v1/me	Current user profile + enrollment summary + active level.
PATCH	/v1/me	Update mutable profile fields (phone, cell group, timezone) with <code>row_version</code> check.
POST	/v1/me/onboarding	Submit the baseline intake (DOB, year of salvation, baptism, cell) + literacy quiz; instantiates enrollment at L1-M1.
POST	/v1/me/devices	Register a device + push token; returns a <code>device_id</code> for sync cursors.

Curriculum & content

Method	Path	Purpose
GET	/v1/levels	The five levels with themes and pass marks.
GET	/v1/levels/{n}/modules	Modules for a level. <i>Gated</i> : only returns unlocked module bodies; locked ones return metadata + <code>locked:true</code> .
GET	/v1/modules/{id}	Full lesson content + signed media URL. 409 GATE_LOCKED if not unlocked for caller.
GET	/v1/scripture	Sanitised YouVersion passage (server-side fetch + HTML strip) by ref + version + language.
PUT	/v1/admin/modules/{id}	<i>Admin</i> . Edit lesson content/pass mark; writes a new <code>module_versions</code> row.
POST	/v1/admin/modules/{id}/questions	<i>Admin</i> . Add/replace question-bank items.

Progress, assessment & reflections

Method	Path	Purpose
POST	/v1/modules/{id}/complete	Mark module complete (idempotent via <code>client_mutation_id</code>); re-evaluates gating.
GET	/v1/modules/{id}/quiz	Assemble a randomised quiz from the module bank; returns served question set.
POST	/v1/modules/{id}/quiz/attempts	Submit answers; server scores, writes attempt + per-answer rows, returns pass/fail.
POST	/v1/levels/{n}/exam/attempts	Submit the level exit exam; $\geq$ pass mark required to proceed.
POST	/v1/levels/{n}/reflection	Submit the descriptive reflection; enqueues to the review queue.
POST	/v1/events/{id}/attendance	Record a QR check-in (idempotent via <code>client_scan_id</code>); validates the scan token.

Multiplier / admin portal

Method	Path	Purpose
GET	/v1/cohorts/{cell_id}/members	Cohort members sorted by <code>e_score</code> asc, with band; scoped to caller's <code>leader_assignments</code> .
GET	/v1/members/{id}/engagement	A member's H/C/Ai breakdown + recent signal history.
GET	/v1/reviews?state=pending	The reflection-review queue for the caller's scope.
POST	/v1/reviews/{id}/decision	Approve/reject a reflection (+notes). Approval flips the level and triggers certificate issuance.
POST	/v1/relationships	Log a multiplier – disciple edge in the relationship tree.
PATCH	/v1/members/{id}/milestones	Record external milestones (water baptism verified, etc.).

Giving, products & certificates

Method	Path	Purpose
POST	/v1/giving/intents	Create a Stripe PaymentIntent for tithe/offering/giving (idempotent); returns client secret.
GET	/v1/giving/history	The caller's own giving/purchase ledger (read of transactions).
GET	/v1/products	Purchasable media catalogue.
POST	/v1/products/{id}/purchase	Buy media; on success grants access via purchases.
GET	/v1/certificates	The caller's issued certificates with download URLs.
GET	/v1/verify/{code}	Public. Verify a certificate by its printed code; returns issuance facts + validity.

Webhooks & sync (system)

Method	Path	Purpose
POST	/v1/webhooks/stripe	Stripe event receiver; HMAC-verified + idempotent (PRD middleware, §1.10 Flow C).
POST	/v1/sync/pull	Delta pull: client cursors in → changed rows + tombstones + new cursors out.
POST	/v1/sync/push	Replay the offline mutation queue; per-mutation results (§3.6).

3.4 Representative request / response bodies

Submit a quiz attempt

```
POST /v1/modules/3f.../quiz/attempts
Authorization: Bearer <jwt>
{ "client_mutation_id": "cla9-...uuid",
  "answers": [ { "question_id": "q1...", "given_answer": "B" },
               { "question_id": "q2...", "given_answer": "True" } ] }

200 OK
{ "attempt_id": "att...", "score_achieved": 84.00, "is_passed": true,
  "pass_mark": 70.00, "unlocked_next_module_id": "mod..." }
```

Cohort read (multiplier dashboard)

```
GET /v1/cohorts/ab12.../members?sort=e_score&order=asc&limit=50
200 OK
{ "data": [
  { "user_id": "u9...", "full_name": "...", "e_score": 0.45, "band": "watch",
    "h_score": 0.30, "c_score": 0.62, "a_score": 0.40, "window_end": "2026-06-04",
    "last_active_days_ago": 14 },
  { "user_id": "u3...", "e_score": 0.58, "band": "steady", ... } ],
  "next_cursor": "eyJlIjowLjU4fQ==" }
```

Create a giving intent

```
POST /v1/giving/intents
Idempotency-Key: give_2026-06-05_u9_001
{ "fund": "tithe", "amount_minor": 50000, "currency": "KES" }

201 Created
{ "transaction_id":"txn...", "stripe_client_secret":"pi...secret...",
  "status":"requires_action" } // client completes with Stripe Elements
```

NOTE ON MONEY PATH

The card number never reaches this API. The client tokenises with Stripe Elements and confirms directly against Stripe; our server only ever sees the PaymentIntent id and, later, the verified webhook. This is what keeps the platform at PCI-DSS SAQ-A (§5.6).

3.5 Webhook contract (Stripe)

The receiver is the PRD's `verifyStripeWebhookIdempotent` middleware verbatim in intent: HMAC-SHA256 signature check, then a `SELECT ... FOR UPDATE NOWAIT` against `processed_webhooks` inside a transaction so a duplicate delivery is dropped with `200` and a genuine first delivery posts the ledger entries. Handled event types: `payment_intent.succeeded` → mark transaction succeeded + post double-entry ledger + grant purchase; `payment_intent.payment_failed` → mark failed; `charge.refunded` → reverse ledger. Unknown types are acknowledged and ignored.

3.6 Offline batch-sync contract

Pull

```
POST /v1/sync/pull
{ "device_id":"dev...",
  "cursors": { "modules": 90412, "module_progress": 90455, "engagement": 90460 } }

200 OK
{ "changes": {
  "modules":      [ { "op":"upsert", "row": { ... } } ],
  "module_progress": [ { "op":"upsert", "row": { ... } } ],
  "engagement":    [ { "op":"upsert", "row": { "e_score":0.45, "band":"watch" } } ] },
  "tombstones": { "products": [ "prod_old_id" ] },
  "cursors": { "modules": 90480, "module_progress": 90479, "engagement": 90480 } }
```

Push (replay)

```
POST /v1/sync/push
{ "device_id": "dev...",
  "mutations": [
    { "mutation_id": "c1a9-...", "seq": 482, "domain": "module_progress",
      "op": "complete", "payload": { "module_id": "...", "completed_at": "...+03:00" } },
    { "mutation_id": "b3f2-...", "seq": 483, "domain": "attendance",
      "op": "scan", "payload": { "event_id": "svc-2026-06-01", "scanned_at": "...+03:00" } } ] }

200 OK
{ "results": [
  { "mutation_id": "c1a9-...", "status": "applied", "server_version": 7 },
  { "mutation_id": "b3f2-...", "status": "duplicate" } ] }
```

Mutations are applied in `seq` order within a request; a `rejected` result carries an error `code` (e.g. `GATE_LOCKED`) and the client surfaces it and drops the mutation. The whole push is safe to retry — applied/duplicate results are stable across retries because of the idempotency keys persisted in the schema (§2.2).

3.7 OpenAPI fragment

The full contract lives in a versioned `openapi.yaml` validated in CI. An illustrative slice:

```
paths:
  /v1/modules/{id}/quiz/attempts:
    post:
      summary: Submit a quiz attempt
      security: [ { bearerAuth: [] } ]
      parameters: [ { name: id, in: path, required: true, schema: { type: string, format: uuid } } ]
      requestBody:
        required: true
        content: { application/json: { schema: { $ref: '#/components/schemas/QuizSubmission' } } }
      responses:
        '200': { description: Scored, content: { application/json: { schema: { $ref: '#/components/schemas/QuizResult' } } } }
        '409': { description: Module locked, content: { application/json: { schema: { $ref: '#/components/schemas/Error' } } } }
components:
  schemas:
    QuizSubmission:
      type: object
      required: [client_mutation_id, answers]
      properties:
        client_mutation_id: { type: string, format: uuid }
        answers:
          type: array
          items: { type: object, required: [question_id, given_answer],
            properties: { question_id: {type: string, format: uuid}, given_answer: {type:
string} } }
      securitySchemes:
        bearerAuth: { type: http, scheme: bearer, bearerFormat: JWT }
```

4. Infrastructure Design

Infrastructure exists to honour three PRD promises: reliable behaviour on weak networks, hundreds of thousands of concurrent actions without deadlock, and smooth global media. The design anchors the write primary close to the largest user segment, pushes reads to replicas and caches, and pushes bytes to the CDN edge.

4.1 Region & latency strategy

The membership is global but heavily African, so a US/EU primary would tax the majority. The write primary is anchored in **af-south-1 (Cape Town)**, with read replicas and edge presence added where membership concentrates (e.g. a EU/ME replica for diaspora). Media is independent of region entirely — Cloudinary's global edge serves it close to every viewer.

Layer	Placement
Write primary (PostgreSQL)	Single home region, multi-AZ for HA.
Read replicas	Home region + one diaspora region; dashboards/curriculum read from nearest.
Static & video	Cloudinary CDN, global edge, capped at 720p/30fps per PRD \$7.3.
API gateway	Anycast/edge-terminated TLS to shorten the handshake from far clients.

4.2 Network topology

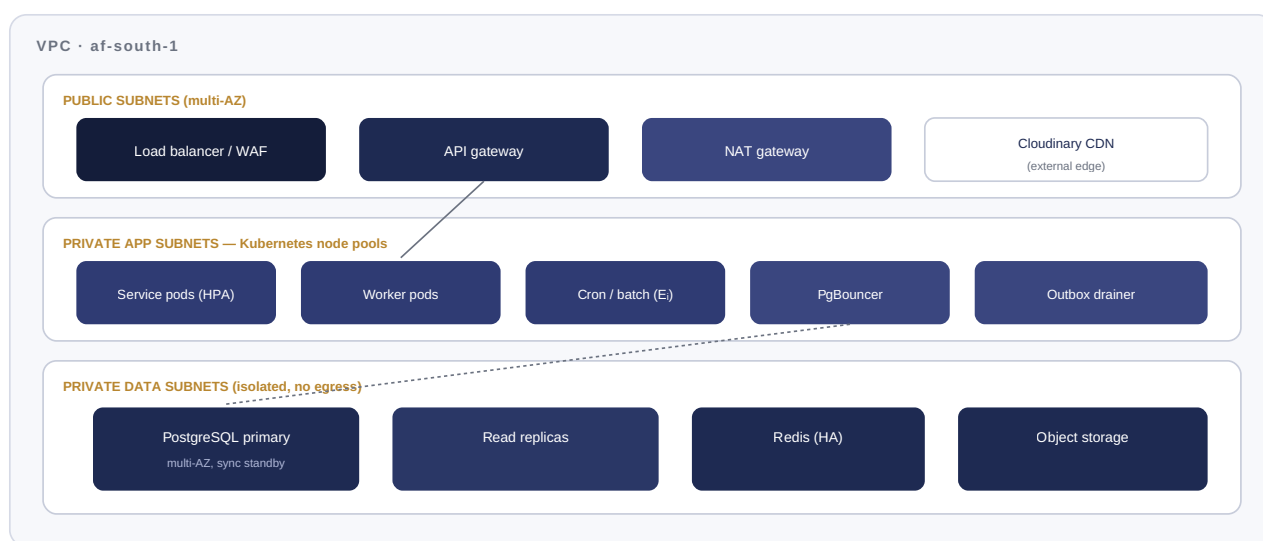


Figure 2 — Three-zone VPC. The data subnets have no internet egress; only the app tier reaches them, and only the gateway/LB faces the public internet.

4.3 Compute & orchestration

- **Containerised microservices on Kubernetes** (managed control plane). Each service is a Deployment with a Horizontal Pod Autoscaler keyed on CPU and request latency; minimum replicas ≥ 2 across AZs so a node loss never takes a service to zero.
- **Separate node pools** for latency-sensitive API pods vs. CPU-heavy batch/worker pods, so the nightly E_i recompute cannot starve live traffic.

- **Stateless pods only.** No service holds session or sync state in memory; this is what lets HPA add and drop pods freely under load.
- **PodDisruptionBudgets + rolling deploys** keep capacity during updates; readiness probes gate traffic until a pod's DB pool and caches are warm.

4.4 Data infrastructure

Component	Configuration
PostgreSQL	Managed, multi-AZ with a synchronous standby for zero-RPO failover; asynchronous read replicas for scale-out reads.
Connection pooling	PgBouncer in transaction mode between the (many) stateless pods and the (finite) backend connections — the deadlock/exhaustion guard the PRD's scalability goal needs.
Redis	Managed, HA with automatic failover; separate logical DBs for sessions, cache, and the delayed-job queue.
Object storage	Versioned bucket for certificate PDFs and backups; lifecycle rules archive cold objects; server-side encryption on.
Partition maintenance	A scheduled job provisions next-month partitions for <code>interaction_events</code> / <code>change_log</code> and prunes > 13 months.

4.5 Media & CDN

Per PRD §7.3, transcode pipelines hard-cap at 720p/30fps for thermal and battery reasons; HLS/DASH adaptive bitrate is brokered by the Media service issuing signed Cloudinary URLs. Pre-resolution of stream endpoints during view-load (PRD §7.3) means tapping "Play" feels instant. The app and API never proxy video bytes — they only hand out signed manifest URLs.

4.6 CI/CD pipeline

1. **Commit** → lint, type-check, unit tests, OpenAPI & migration validation against an ephemeral Postgres.
2. **Build** → container images, signed and scanned for CVEs; SBOM recorded.
3. **Staging deploy** → migrations applied, integration + contract tests, smoke tests against the live-like environment.
4. **Production** → migration gate, then canary rollout (small traffic %) watched on error-rate/latency SLOs; auto-rollback on breach.
5. **Mobile** → CodePush-style OTA for JS-layer fixes; native binary releases through App Store / Play Console with staged rollout.

4.7 Observability & SLOs

Signal	Tooling & target
Logs	Structured JSON, correlation by <code>X-Request-Id</code> , centralised and retention-tiered.
Metrics	RED (rate/errors/duration) per service + USE for nodes; dashboards per domain.
Tracing	Distributed traces across gateway → service → DB to localise tail latency.
Alerting SLOs	API availability $\geq 99.9\%$ monthly; read p95 < 300 ms; write p95 < 600 ms; sync-push success $\geq 99.5\%$; payment-webhook processing < 5 s p99.
Error tracking	Client + server exception capture with release tagging.

4.8 Scaling & capacity

The concurrency target is met by keeping writes small and reads cheap. Capacity reasoning (full arithmetic in Appendix A): a service burst around Sunday produces a spike of attendance scans and habit events; because those are tiny append-only inserts, batched through sync, the write path stays well within a single well-provisioned primary's headroom, and the read-heavy dashboard load is absorbed by replicas + Redis. Autoscaling reacts to latency, not just CPU, so the system grows ahead of saturation rather than after it.

4.9 Disaster recovery & backups

Measure	Policy
RPO	≈ 0 for committed transactions via the synchronous standby; ≤ 5 min for cross-region copy.
RTO	< 30 min: automated standby promotion + DNS/endpoint cutover.
Backups	Automated daily snapshots + continuous WAL archiving enabling point-in-time recovery; 30-day retention.
Restore drills	Quarterly restore-to-scratch test; a backup that has never been restored is not a backup.
Object storage	Versioned + cross-region replicated for certificates (legal/credential durability).

4.10 Environments & promotion

Three isolated environments — **dev** (ephemeral, per-branch), **staging** (production-shaped, anonymised data), **production** — with infrastructure defined as code so environments are reproducible and drift is detectable. Secrets are never shared across environments; staging uses Stripe/KingsChat sandbox credentials only.

4.11 Cost guardrails

Read replicas and CDN absorb the expensive traffic so the primary stays modestly sized; batch compute runs on spot/low-priority capacity during off-peak windows; object lifecycle rules tier cold data down. Budget alarms fire on anomalous spend (e.g. a runaway transcode or egress spike) before the invoice does.

5. Security Model

The platform holds sensitive personal data (names, phone numbers, dates of birth, baptism status) for a global membership that includes minors, and it touches money. The security model is built on defence-in-depth, least privilege, and keeping the highest-risk data (cards) entirely off our infrastructure.

5.1 Security posture & principles

Principle	Applied as
Least privilege	Every role, service account, and DB grant gets the minimum it needs; multipliers see only their assigned cells.
Defence in depth	Edge WAF + gateway authN + service authZ + row-level scoping + encrypted storage — no single control is load-bearing alone.
Minimise blast radius	Cards never touch us (PCI SAQ-A); secrets in a manager, not env files; data subnets have no internet egress.
Auditable by default	Privileged actions (reflection approval, role change, ledger posting) write immutable <code>audit_log</code> rows.
Secure the weak network, not just the strong one	Offline tokens and cached data are encrypted on-device because the device, not the server, is the exposed surface in the field.

5.2 Threat model (STRIDE)

Threat	Scenario	Mitigation
Spoofing	Forged identity / stolen token reuse.	Signed short-lived JWTs; refresh rotation with reuse detection revoking the family; SSO via trusted IdPs.
Tampering	Faked quiz pass, forged certificate, altered ledger.	Server-authoritative scoring & gating; tamper-evident signed certificates; append-only double-entry ledger + audit log.
Repudiation	"I never approved that" / "I never gave."	Audit log with actor + timestamp; payment trail anchored to Stripe events and webhook ledger.
Information disclosure	Leak of minors' PII or another cell's data.	Row-level scoping by <code>leader_assignments</code> ; field-level PII encryption; TLS everywhere; no PII in logs.
Denial of service	Auth/payment flooding; sync abuse.	Per-user/IP rate limits with stricter buckets on auth/payment/sync; WAF; autoscaling with circuit breakers.
Elevation of privilege	Student calling admin endpoints; replay of webhook.	RBAC enforced server-side on every request; idempotent HMAC-verified webhooks; no client-trusted role claims.

5.3 Authentication

- **Federated sign-in.** OAuth 2.0 / OIDC with KingsChat (primary identity), Google, and Apple. The authorization-code exchange happens server-side; we validate the IdP token, then mint our own session — we never trust a client-asserted identity.
- **Token model.** A short-lived (15-min) access JWT carries the user id, role, and congregation scope; a long-lived refresh token (stored only as a SHA-256 hash, never plaintext) lives in `refresh_tokens` as a rotating family.
- **Rotation & reuse detection.** Each refresh issues a new token and invalidates the old; if a previously-used token is presented again (a sign of theft), the entire family is revoked and the user must re-authenticate.

- **Admin step-up.** SuperAdmin and financial-config actions require MFA; admin sessions are shorter and re-prompt for sensitive operations.
- **On-device token storage.** Access/refresh tokens are held in the OS secure enclave (iOS Keychain / Android Keystore), never in plain SQLite or async-storage.

5.4 Authorization (RBAC + scoping)

Coarse role from `users.role`, then fine-grained resource scoping from `leader_assignments`. Every request is checked server-side; the client's role claim is convenience, never authority.

Capability	Student	Instructor / Multiplier	Admin	SuperAdmin
Own progress / quizzes / giving	✓	✓	✓	✓
View assigned-cohort engagement	—	✓ (own cells)	✓ (branch)	✓ (all)
Approve reflections	—	✓ (own cells)	✓	✓
Edit curriculum / question banks	—	—	✓	✓
Manage funds / financial config	—	—	— (view)	✓ (MFA)
Assign roles / leaders	—	—	—	✓ (MFA)

Scoping is enforced in the query layer: a multiplier's cohort read joins through `leader_assignments`, so a member outside their cells is not merely hidden in the UI — the data never leaves the database for that caller. A `403 FORBIDDEN_SCOPE` is returned for out-of-scope ids.

5.5 Data protection & key management

Control	Detail
In transit	TLS 1.2+ on every hop, including inside the mesh; HSTS at the edge.
At rest	Full-disk/volume encryption on DB, Redis, and object storage via managed KMS.
Field-level PII	Phone number and other direct identifiers encrypted at the application layer with a KMS-managed key, so a raw DB dump does not expose them in clear text.
Password hashing	Argon2id for the rare password accounts; the majority are SSO-only with no stored secret.
Key management	Keys in a managed KMS/HSM; rotation scheduled; the certificate-signing key is separate and access-logged.
Logging hygiene	Structured logs scrub PII, tokens, and card-adjacent fields; correlation uses opaque request ids.

5.6 Payment security

- **PCI-DSS SAQ-A.** Card data is captured and tokenised entirely by Stripe Elements in the client; it never traverses our API or storage. We persist only Stripe ids, our own ledger, and verified webhook records — keeping the platform in the lightest PCI scope.
- **Webhook integrity.** Every Stripe webhook is HMAC-SHA256 verified with a constant-time comparison before processing, then de-duplicated against `processed_webhooks` under a row lock (PRD middleware; §3.5) so replays and double-deliveries cannot double-post the ledger.
- **Idempotent giving.** Client-supplied `Idempotency-Key` on intent creation prevents a retried tap on a flaky network from creating two donations.
- **No offline money.** The giving flow hard-blocks offline; financial intent is never queued for replay.

5.7 Device & offline security

- **Encrypted local store.** The SQLite cache is encrypted (SQLCipher or platform equivalent); cached curriculum and progress are protected if a device is lost.
- **Secure token vault.** Tokens in Keychain/Keystore, not in the encrypted DB, so a DB compromise still does not yield credentials.
- **Tamper signals.** The app detects jailbreak/root and degrades gracefully (e.g. refuses to cache certain content) rather than trusting a compromised runtime.
- **Sync authenticity.** Every push/pull is authenticated and bound to a registered `device_id`; server-authoritative merge means a tampered local store cannot fabricate progress or unlock gated content.

5.8 API hardening (OWASP API Top 10)

Risk	Control
Broken object-level authz (BOLA)	Ownership/scope checked on every id-addressed resource; never "trust the id in the path".
Broken authentication	Short tokens, rotation, reuse detection, secure storage (§5.3).
Excessive data exposure	Explicit response DTOs; the API never serialises raw DB rows or internal fields.
Resource exhaustion	Pagination caps, payload-size limits, per-bucket rate limits, query timeouts.
Mass assignment	Allow-listed writable fields per endpoint; <code>role</code> / <code>congregation_id</code> are never client-writable.
Injection	Parameterised queries only; input validated against the OpenAPI schema at the edge.
Improper inventory	Single versioned API surface; deprecated versions sunset on a published schedule.

5.9 Privacy, minors & compliance

MINORS ARE FIRST-CLASS HERE

Because intake captures date of birth, the schema derives an `is_minor` flag. Any member under 18 is treated as a protected data subject: their personal data is minimised, never used for the public certificate-verification lookup beyond a name/level, and engagement data is visible only to their assigned pastoral leaders — never broadly searchable.

- **Kenya Data Protection Act (2019).** The design supports its core obligations — lawful basis, data minimisation, purpose limitation, subject rights (access/rectification/erasure), and breach notification. *This is an engineering posture, not a legal opinion; a qualified data-protection advisor should validate processing of minors' data and any cross-border transfer before launch.*
- **Consent for minors.** Onboarding for under-18 members should capture guardian consent; the platform records the consent basis and restricts processing accordingly.
- **Data subject rights.** Soft-delete (`deleted_at`) plus a hard-erasure job satisfies erasure requests while preserving the integrity of the financial ledger (which is retained for accounting obligations and anonymised on erasure).
- **Data minimisation & retention.** Raw interaction events are pruned after 13 months (only the derived engagement snapshot persists); logs are retention-tiered; backups age out at 30 days.
- **Cross-border transfer.** Anchoring the primary in-region reduces transfer exposure; any diaspora replica is governed by a transfer assessment.

5.10 Secrets, audit & monitoring

- **Secrets management.** All credentials (DB, Stripe, KingsChat, signing keys) live in a managed secrets store with rotation and access logging — never in code, images, or env files committed to the repo.
- **Audit trail.** The `audit_log` captures every privileged action with actor, target, and time; it is append-only and shipped to immutable storage.
- **Security monitoring.** Alerts on anomalous auth failures, refresh-reuse events, webhook-signature failures, and out-of-scope access attempts feed the on-call rotation.

5.11 Incident response & vulnerability management

- **Dependency & image scanning** in CI; CVE thresholds block release; SBOMs retained per build.
- **Patching cadence** for base images and managed services; emergency patch path for criticals.
- **IR runbook:** detect → contain (revoke tokens/keys, isolate) → eradicate → recover (restore from PITR if needed) → notify (per DPA timelines) → post-mortem. Quarterly tabletop exercises keep it real.

A. Capacity & Concurrency Math

The PRD asks the backend to absorb "*hundreds of thousands of concurrent international member actions without data degradation or deadlocks.*" This appendix shows the arithmetic behind the claim in §4 that this is reachable on a single well-provisioned primary plus replicas. All figures are an **illustrative design model** — they are sized to be revised once real telemetry exists, not presented as measured fact.

A.1 Population model

Quantity	Value	Basis
Registered members (design target)	500,000	Global-church scale ceiling for v1 sizing.
Daily active members (DAU)	~150,000	30% of registered — typical for a habit-formation app with daily nudges.
Peak concurrent sessions	~12,000	~8% of DAU active within a 2-hour service/evening window.
Interaction events / active member / day	~8	Lesson opens, scripture reads, video plays, quiz actions.

A.2 Write path — the part that must not deadlock

Every write the PRD generates is a **small, append-only insert**, and most arrive batched through the sync engine rather than as live round-trips. That is the whole reason the concurrency target is comfortable rather than frightening.

Write source	Burst arithmetic	Peak rate
Attendance QR scans (50,000-person service checking in over ~15 min)	$50,000 \div (15 \times 60)$	~55/s avg → ~165/s peak
Habit / interaction events (60% of 1.2M daily events land in a 3-hour window)	$720,000 \div (3 \times 3600)$	~67/s avg → ~200/s peak
Quiz attempts, progress, giving (low volume, spiky)	—	~35/s peak
Combined sustained write peak	—	~400 inserts/s

WHY THIS IS SAFE

A single well-provisioned PostgreSQL primary sustains several thousand small inserts per second. A ~400/s peak of tiny append-only rows — partitioned monthly (§2.4) so the hot index stays small — leaves the primary running at a fraction of capacity. Deadlock risk is avoided structurally: these inserts touch independent rows, take no broad locks, and the one place contention *could* occur (the webhook ledger) uses `FOR UPDATE NOWAIT` to fail fast rather than block (§3.5).

A.3 Read path — absorbed away from the primary

Read source	Peak estimate	Served from
Dashboard loads (12,000 sessions, refresh ~every 30s)	~400/s	Redis snapshot + read replica
Curriculum / module content	~300/s	Replica + Redis (immutable, cache-friendly)
Scripture text (YouVersion)	~250/s	Sanitised + cached at edge; never re-fetched per read
Video lessons	all sessions	CDN only — never touches a service or the DB
Cohort telemetry (multiplier portal)	~50/s	Read replica against the engagement snapshot table

Reads outnumber writes by roughly an order of magnitude, exactly the asymmetry §1.1 designs for: the primary takes only the ~400/s of small writes, while the much larger read volume is fanned out to replicas, Redis, and the CDN.

A.4 Connection & pool sizing

Stateless services scale horizontally, which would normally explode the database connection count. PgBouncer in transaction-pooling mode collapses it: thousands of client-side connections multiplex down to a small, fixed server-side pool, keeping the primary's connection slots safe regardless of how many pods are running.

Layer	Sizing
Application pods (peak)	Autoscaled on p95 latency; tens of pods across services.
Client connections into PgBouncer	~2,000
Server connections PgBouncer → primary	~60 (transaction mode)
Read replicas	≥2, behind the same pooler, taking all dashboard/curriculum/telemetry reads.

A.5 Storage growth & the engagement batch

- **Interaction events:** ~1.2M rows/day × ~200 B ≈ 240 MB/day raw, ~7 GB/month. With the 13-month prune (§5.9) the partitioned table stabilises near ~90 GB — small, and never the hot path because reads go through the engagement snapshot, not the raw events.
- **Nightly Ei recompute:** aggregates the trailing 30-day window for ~150,000 active enrollments into the snapshot table. Because each member's three components are simple windowed aggregates over an indexed, partitioned table (§2.5), the batch completes in minutes; intraday deltas are folded in incrementally (§1.8) so dashboards never wait on the batch.

BOTTOM LINE

The headline "hundreds of thousands of concurrent actions" resolves, under load, to a few hundred small inserts per second and a read tier that never touches the primary. The constraint that actually governs scale is *keeping writes small and reads off the primary* — which the architecture does by construction.

B. Configuration & Environment

Everything in this specification that is a *number* rather than a *structure* is collected here so it lives in one place, under change control, instead of being scattered through code. Two tables: the runtime environment each service expects, and the tunable business/operational constants. The constant values match exactly what the body of this document specifies — this table is the single source of truth for them.

B.1 Service environment variables

Secrets are *referenced* here by name only; their values live in the managed secrets store (§5.10) and are never committed.

Variable	Purpose
DATABASE_URL	Primary Postgres DSN (via PgBouncer).
DATABASE_REPLICA_URL	Read-replica DSN for dashboards & telemetry.
REDIS_URL	Session/cache cluster endpoint.
JWT_SIGNING_KEY (secret)	Active key for access-token signing; rotated.
JWT_ACCESS_TTL	Access-token lifetime — 900 (15 min).
REFRESH_TTL	Refresh-token lifetime; rotating with reuse detection.
KINGCHAT_OIDC_ISSUER / _CLIENT_ID / _SECRET (secret)	KingsChat SSO (OIDC) validation.
OAUTH_GOOGLE_* / OAUTH_APPLE_* (secret)	Secondary OAuth providers.
STRIPE_SECRET_KEY / STRIPE_WEBHOOK_SECRET (secret)	Payments + HMAC webhook verification (§3.5).
YOUVERSION_APP_KEY (secret) / YOUVERSION_LANGUAGE_RANGES	Scripture API key and mandatory language_ranges to avoid 422s.
CLOUDINARY_URL (secret)	Media transcode + CDN credentials.
CERT_SIGNING_KEY (secret)	Certificate signature key (tamper-evident PDFs, §B.2).
PUSH_PROVIDER_KEY (secret)	Push/email nudge dispatch.
AWS_REGION	af-south-1 (Cape Town) primary.
LOG_LEVEL / OTEL_EXPORTER_OTLP_ENDPOINT	Structured logging + traces to the observability stack.

B.2 Tunable constants (single source of truth)

Constant	Value	Defined in
Engagement weight — habits <code>w_h</code>	0.40	\$1.8
Engagement weight — curriculum <code>w_c</code>	0.35	\$1.8
Engagement weight — attendance <code>w_a</code>	0.25	\$1.8
Scoring window	30 days (rolling)	\$1.8
Habit normaliser (active-day target)	20 days	\$1.8
Curriculum denominator	45 modules	\$1.8
Band — thriving / watch / at-risk	≥ 0.75 / mid / < 0.40	\$1.8
Module quiz pass mark	70%	\$2.2, PRD
Level exam pass mark	80%	\$1.9, PRD
Video transcode cap	720p @ 30fps	\$4.5, PRD
Access-token TTL / refresh model	15 min / rotating + reuse detection	\$5.3
Pagination default / max page size	50 / 200	\$3.1
Interaction-event retention	13 months	\$5.9
Backup retention / PITR window	30 days	\$4.7
SLO — availability / read p95	99.9% / < 300 ms	\$4.6
DR — RPO / RTO	≈ 0 / < 30 min	\$4.7

CHANGE DISCIPLINE

Changing any value here is a product/operational decision, not a code change: the weights and pass marks are read from configuration so a curriculum or policy adjustment never requires a redeploy, and every change is captured in `change_log` (§2) for auditability.

C. Assumptions & Open Questions

This specification closes the gaps the PRD left implied. A few decisions, though, sit above the engineering line — they need a product owner, a legal advisor, or a partner confirmation rather than an architect. They are listed here in full so nothing is silently assumed. **None blocks the build; each shapes it.**

C.1 Load-bearing assumptions

Assumption	If it turns out false...
Traffic is read-heavy with bursty write spikes around service times.	The primary sizing in Appendix A would need revisiting; the structural approach (small writes, replica reads) still holds.
KingsChat issues an OIDC-compatible token we can validate server-side.	If it is a proprietary scheme, the Identity service needs a KingsChat-specific verification adapter rather than the generic OIDC path (§5.3).
"Immutable" certificate means tamper-evident (cryptographically signed + verifiable), not blockchain-anchored.	A ledger-anchored credential would add an external dependency and cost; the signed-PDF + verification-code design (§2, §5) is assumed sufficient.
Money is online-only ; no offline giving is queued on-device.	Confirmed design stance (§5.6) — this is an assumption made deliberately, not pending.
YouVersion permits caching sanitised verse text within license terms.	If not, scripture reads cannot be edge-cached and the read estimate in A.3 rises.

C.2 Open questions for stakeholders

Question	Owner	Why it matters
Guardian-consent flow for under-18 members	Product + Legal	Drives onboarding UX and the consent-basis fields (§5.9); required before processing minors' data.
Diaspora read-replica jurisdiction	Legal	Any out-of-region replica needs a cross-border transfer assessment under the DPA (§5.9).
Stripe settlement entity & currency (KES vs multi-currency)	Finance	Determines the Stripe account region and how the ledger records currency (§2 funds/ledger).
AI-assisted reflection grading	Product	PRD defers it to manual review in v1; the <code>reflection_reviews</code> queue is built to accept an automated pre-scorer later without schema change.
Push/email provider choice & the 12-nudge cadence cap	Product	Affects notification-preference modelling and per-user rate limits on nudges.
Final concurrency SLA & the real DAU/peak ratios	Eng + Product	Replaces the illustrative Appendix A model with measured targets once a pilot cohort exists.
Data-retention sign-off (13-month prune, ledger anonymisation)	Legal	The engineering posture in §5.9 needs a qualified data-protection advisor's confirmation before launch.

CLOSING NOTE

This document deliberately separates what the engineering can decide (now decided, end-to-end: architecture, schema, API, infrastructure, security) from what the ministry and its advisors must decide. The former is build-ready. The latter is surfaced here so it is chosen on purpose rather than by default — which is, in the end, the same discipline the platform itself applies to discipleship: make the next step explicit, and never let anyone drift through a gate they didn't actually pass.